

Atelier — Classification de Spectres Raman par Machine Learning

Durée : 1h30 **Public :** Débutants en machine learning (chimistes, biologistes, étudiants en sciences) **Prérequis :** Notions de base en Python ; notions générales de spectroscopie

1. Objectifs de l'atelier

À l'issue de cet atelier, vous serez capable de :

1. **Décrire** le pipeline classique d'un projet de machine learning supervisé.
 2. **Identifier** le rôle de chaque étape : préparation des données, séparation train/validation/test, normalisation.
 3. **Comprendre** l'architecture d'un perceptron multicouche (MLP) et le rôle de ses composants (couches, fonctions d'activation, nombre de paramètres).
 4. **Interpréter** les principales métriques d'évaluation (accuracy, precision, recall, F1, AUC, matrice de confusion).
 5. **Reconnaître** les phénomènes d'underfitting et d'overfitting à partir des courbes d'apprentissage.
 6. **Évaluer** l'impact des hyperparamètres clés : architecture, learning rate, batch size, taille du dataset.
-

2. Choix du support selon le niveau

Deux supports complémentaires sont disponibles. **Un seul est utilisé** durant l'atelier, en fonction du profil des participants.

Option A — Application Streamlit (recommandée pour débutants complets)

- **Fichier :** `app_raman_classification.py`
- **Lancement :** `streamlit run app_raman_classification.py`
- **Avantages :**
 - Aucune ligne de code à écrire ni à modifier.
 - Interface graphique avec onglets « Exploration » et « Entraînement ».
 - Les hyperparamètres se règlent via la barre latérale (sliders, menus).
 - Visualisation en temps réel des courbes d'apprentissage.
- **Pour qui ?** Participants n'ayant jamais codé, ou souhaitant se concentrer sur les concepts plutôt que sur la syntaxe Python.

Option B — Notebook Jupyter (recommandée pour participants avec bases Python)

- **Fichier** : `Raman_Classification_Binaire_enrichi.ipynb`
- **Lancement** : `jupyter notebook` ou ouverture dans VS Code / Jupyter-Lab
- **Avantages** :
 - Lecture pas-à-pas du code commenté.
 - Possibilité de modifier les paramètres directement dans les cellules.
 - Vue détaillée de chaque étape (chargement, prétraitement, modèle, boucle d'entraînement).
- **Pour qui ?** Participants à l'aise avec Python ou désireux de voir le code « sous le capot ».

Note pour l'animateur : la grille de questions ci-dessous est applicable aux deux supports. Les références au code ne concernent que l'option B.

3. Le contexte scientifique

3.1 Les données

Le dataset contient des **spectres Raman** de milieux de culture cellulaire :

- **Classe 0 — Normal** : cellules saines (fibroblastes, mélanocytes, milieux DMEM).
- **Classe 1 — Cancer** : cellules cancéreuses (mélanomes).

Chaque spectre est constitué de **2090 points d'intensité** mesurés à des nombres d'onde compris entre 100 et 4278 cm^{-1} .

3.2 Le problème

À partir d'un spectre brut, **prédire automatiquement** si l'échantillon provient d'un milieu cancéreux ou normal. C'est un problème de **classification binaire supervisée**.

3.3 Pourquoi le machine learning ?

- Les différences spectrales entre les deux classes sont **subtiles et réparties sur de nombreuses bandes**.
 - Un expert humain peut difficilement distinguer les classes à l'œil nu.
 - Le ML apprend automatiquement les régions spectrales discriminantes.
-

4. Déroulé sur 1h30

Phase	Durée	Contenu
1 — Introduction	10 min	Présentation des objectifs, du contexte et du support choisi
2 — Exploration des données	15 min	Structure des fichiers, visualisation de spectres, statistiques
3 — Prétraitement et split	10 min	Normalisation, séparation train/val/test
4 — Construction d'un MLP	15 min	Architecture, nombre de paramètres, hyperparamètres
5 — Entraînement	20 min	Lancement, lecture des courbes, interprétation
6 — Évaluation	15 min	Métriques, matrice de confusion, courbe ROC
7 — Synthèse	5 min	Récapitulatif et pistes pour aller plus loin

5. Activités guidées et questions

Les questions sont organisées par phase. **Répondez par écrit** (sur ce document ou un carnet) au fur et à mesure : elles servent de support à la discussion collective en fin de phase.

Phase 2 — Exploration des données

Questions de compréhension Q2.1 Combien de spectres contient le dataset au total ? Combien dans chaque classe ?

Q2.2 Les classes sont-elles équilibrées ? Pourquoi est-ce important pour la classification ?

Q2.3 Combien de points (features) compose chaque spectre ? Que représente ce nombre physiquement ?

Q2.4 En comparant visuellement un spectre Normal et un spectre Cancer, voyez-vous une différence évidente à l'œil nu ? Justifiez.

Question pratique Q2.5 (Streamlit : onglet « Exploration » → section 6 ; Notebook : cellule de visualisation) Sélectionnez un fichier CSV individuel d'une classe Normal puis d'une classe Cancer. Notez deux régions spectrales (en cm^{-1}) où les intensités semblent les plus différentes.

Phase 3 — Prétraitement et séparation des données

Questions de compréhension Q3.1 Quelle est la différence entre l'ensemble d'**entraînement** (train), de **validation** (val) et de **test** ?

Q3.2 Pourquoi ne peut-on **pas** évaluer la qualité finale d'un modèle sur les données d'entraînement ?

Q3.3 À quoi sert la **stratification** lors du split ?

Q3.4 Qu'est-ce que la **standardisation** (z-score) ? Pourquoi est-ce utile avant d'entraîner un réseau de neurones ?

Q3.5 Pourquoi calcule-t-on les paramètres de standardisation **uniquement sur le train** et pas sur l'ensemble du dataset ? (Notion de *data leakage*)

Phase 4 — Construction du MLP

Questions de compréhension Q4.1 Un MLP (Multi-Layer Perceptron) est composé de **couches denses**. Que fait mathématiquement une couche dense ?

Q4.2 À quoi sert la fonction d'**activation ReLU** entre deux couches denses ? Que se passerait-il si on l'enlevait ?

Q4.3 La couche de sortie a **2 neurones** (un par classe). Pourquoi pas un seul neurone ?

Question pratique — Expérimenter avec l'architecture Q4.4 (Streamlit : sidebar « Architecture du Réseau » ; Notebook : modifier `hidden_dims`) Comparez les architectures suivantes en relevant à chaque fois le **nombre de paramètres entraînaibles** :

Architecture	Couches cachées	Nombre de paramètres
Minimaliste	[1]	
Très simple	[64, 32]	
Simple	[128, 64]	
Moyenne	[256, 128, 64]	
Complexe	[512, 256, 128]	

Q4.5 Que constatez-vous ? Quelle couche concentre le plus de paramètres et pourquoi ?

Phase 5 — Entraînement

Questions de compréhension Q5.1 Une **epoch** correspond à un passage complet sur les données d'entraînement. Qu'est-ce qu'un **batch** ?

Q5.2 Le **learning rate** contrôle la taille des « pas » d'apprentissage. Que se passe-t-il s'il est trop grand ? Trop petit ?

Q5.3 Qu'est-ce que la **loss** (fonction de coût) ? Doit-elle augmenter ou diminuer au cours de l'entraînement ?

Questions pratiques — Expérimenter Q5.4 Lancez un premier entraînement avec les paramètres par défaut suivants :

- Architecture : **Simple (128-64)**
- Learning rate : **0.001**
- Epochs : **30**
- Batch size : **32**

Notez la **train accuracy finale** et la **val accuracy finale**.

Q5.5 — Underfitting Lancez maintenant un entraînement avec l'architecture **Minimaliste (1)**. Que constatez-vous sur la val accuracy ? Comment interprétez-vous cela ?

Q5.6 — Effet du learning rate Reprenez l'architecture Simple (128-64) et testez deux learning rates extrêmes :

- $lr = 0.00001$ (très petit)
- $lr = 0.1$ (très grand)

Décrivez l'allure des courbes de loss dans chaque cas. Quelle valeur semble la meilleure ?

Q5.7 — Effet de la taille du dataset (Streamlit uniquement, ou modification du split en notebook) Activez l'option « Limiter la taille du dataset d'entraînement » et essayez avec **50 échantillons**, puis **200**, puis **1000**. Que constatez-vous sur la val accuracy ?

Q5.8 — Lecture des courbes Sur l'un de vos entraînements, indiquez :

- La courbe **train accuracy** est-elle au-dessus ou en-dessous de la **val accuracy** ? Pourquoi ?
- Si la train accuracy continue de monter alors que la val accuracy stagne (ou descend), de quel phénomène s'agit-il ?

Phase 6 — Évaluation

Questions de compréhension Q6.1 Qu'est-ce qu'une **matrice de confusion** ? Que représentent les 4 cases ?

Q6.2 Donnez les définitions intuitives suivantes :

- **Accuracy** :
- **Precision** :
- **Recall (sensibilité)** :
- **F1-Score** :

Q6.3 Dans un contexte de diagnostic médical (détection cancer/normal), quelle métrique est la **plus critique** à maximiser : la precision ou le recall ? Justifiez.

Q6.4 Une **AUC = 0.5** correspond à quel comportement du modèle ? Une **AUC = 1.0** ?

Question pratique **Q6.5** Notez les valeurs obtenues sur votre meilleur entraînement :

Métrique	Valeur
Accuracy	
Precision	
Recall	
F1-Score	
AUC	
Faux positifs	
Faux négatifs	

Q6.6 Combien de cas de cancer le modèle a-t-il manqués (faux négatifs) ? Combien de personnes saines a-t-il faussement diagnostiquées comme cancer (faux positifs) ?

6. Synthèse — Ce que vous devez retenir

À la fin de l'atelier, complétez ce résumé avec vos propres mots :

1. **Le pipeline d'un projet ML supervisé comprend** :
2. **Un MLP est un réseau de neurones constitué de** :
3. **Pour éviter le surapprentissage (overfitting), il faut** :
.....
4. **Pour évaluer un modèle de classification, on utilise** :
.....
5. **Le choix des hyperparamètres (learning rate, architecture, batch size) influence** :

7. Pour aller plus loin

Une fois les bases acquises, plusieurs pistes peuvent être explorées :

- **Régularisation** : ajouter du *dropout* ou du *weight decay* pour limiter l'overfitting.
- **Architectures spécialisées** : utiliser un **CNN 1D** pour exploiter la structure séquentielle des spectres.
- **Interprétabilité** : visualiser les régions spectrales les plus discriminantes (poids de la première couche).
- **Comparaison** : opposer le MLP à un **Random Forest** sur les mêmes données (voir la dernière section du notebook).
- **Validation croisée** : remplacer le split fixe par une *k-fold cross-validation* pour une évaluation plus robuste.

8. Glossaire rapide

Terme	Définition courte
Feature	Variable d'entrée du modèle (ici, une intensité à un nombre d'onde donné).
Label	Étiquette de la classe (0 = Normal, 1 = Cancer).
MLP	Multi-Layer Perceptron : réseau de neurones composé de couches entièrement connectées.
Epoch	Un passage complet du modèle sur toutes les données d'entraînement.
Batch	Sous-ensemble de données vu en une seule étape d'optimisation.
Loss	Mesure de l'erreur du modèle, à minimiser.
Optimizer	Algorithme qui met à jour les poids du modèle (ex. Adam, SGD).
Overfitting	Le modèle apprend par cœur le train mais généralise mal.
Underfitting	Le modèle est trop simple pour capter les patterns des données.
ROC / AUC	Courbe et aire mesurant la qualité de discrimination du modèle.

Bonne exploration !